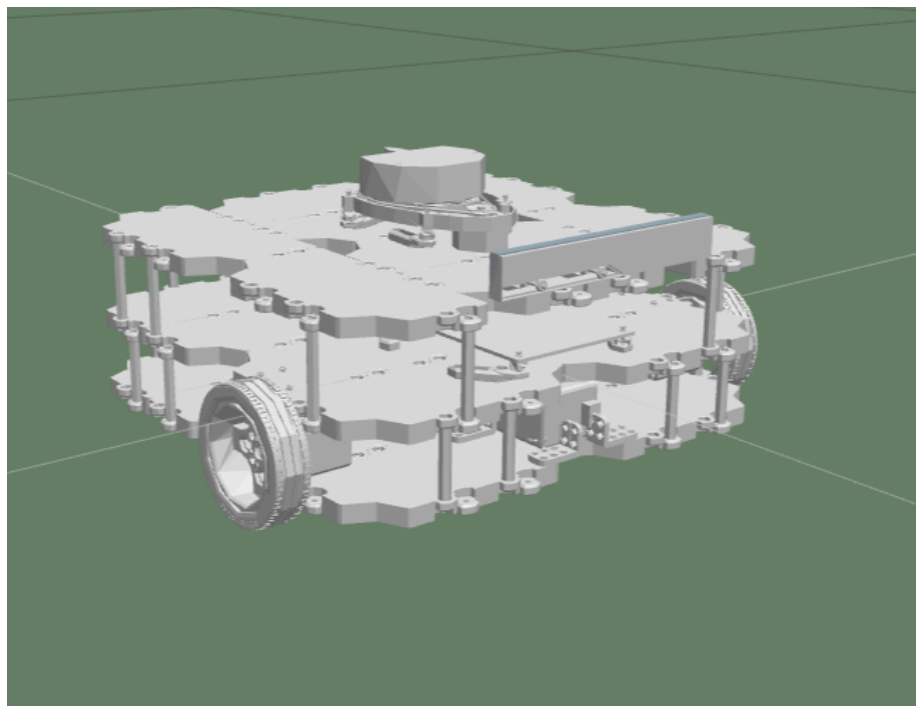


RA: Robòtica Mòbil 2026

Pràctiques

2- Topics de ROS



Dr. Anaís Garrell Zulueta

Departament d'Enginyeria de Sistemes, Automàtica i Informàtica Industrial

CONTINGUTS

[1. Què Aprendre?](#)

[2. Topic Publisher](#)

[3. Messages](#)

[4. Topic Subscriber](#)

[Exercici 1 \(Entregable\)](#)

1. Què Aprendrem?

- Què són els **topics** de ROS i com gestionar-los
- Què és un **publisher** i com crear-ne un.
- Què són els **topic messages** i com funcionen
- Què és un **subscriber** i com crear-ne un

2. Topic Publisher

Exercici 1:

- Creeu un paquet nou anomenat **topic_publisher_pkg_vostrenom**. Quan creeu el paquet, afegiu com a dependències **roscpp** i **std_msgs**.
- Dins de la carpeta **src** del paquet, creeu un fitxer nou anomenat **simple_topic_publisher_vostrenom.cpp**. Dins d'aquest fitxer, copieu el contingut de **simple_topic_publisher.cpp**
- Creeu un fitxer **launch** per llançar aquest codi.
- Feu les modificacions necessàries al fitxer **CMakeLists.txt** i compileu el paquet.
- Executeu el fitxer **launch** per executar el vostre executable

```
#include <ros/ros.h>
#include <std_msgs/Int32.h>

int main(int argc, char** argv) {

    ros::init(argc, argv, "topic_publisher_vostrenom");
    ros::NodeHandle nh;

    ros::Publisher pub = nh.advertise<std_msgs::Int32>("counter", 1000);
    ros::Rate loop_rate(2);

    std_msgs::Int32 count;
    count.data = 0;

    while (ros::ok())
    {
        pub.publish(count);
        ros::spinOnce();
        loop_rate.sleep();
        ++count.data;
    }
}
```

```
}  
  
    return 0;  
}
```

NOTA:

1.- Per fer aquest exercici, simplement pots seguir els mateixos passos què vàrem explicar a la sessió 1.2.

2.- Recordeu que, per crear un paquet amb **roscpp** i **std_msgs** com a dependències, hauríeu d'utilitzar una ordre com la següent:

T1: `$ catkin_create_pkg topic_publisher_pkg_vostrenom roscpp std_msgs`

3.- Les línies a afegir al fitxer CmakeLists.txt podrien ser una cosa així:

```
add_executable(simple_topic_publisher_vostrenom  
src/simple_topic_publisher_vostrenom.cpp)  
add_dependencies(simple_topic_publisher_vostrenom  
${simple_topic_publisher_vostrenom_EXPORTED_TARGETS}  
${catkin_EXPORTED_TARGETS})  
target_link_libraries(simple_topic_publisher_vostrenom  
    ${catkin_LIBRARIES}  
)
```

Llanceu el vostre arxiu **launch**, no passa res? això no és veritat! Acabeu de crear un topic anomenat **/counter** i publicat a través d'ell com un nombre enter que augmenta indefinidament, recordeu llançar el **roscore** en primer lloc.

Un topic és com un canal. Els nodes utilitzen topics per publicar informació per a altres nodes perquè es puguin comunicar.

Podeu esbrinar, en qualsevol moment, el nombre de topics del sistema fent un **rostopic list**. També podeu comprovar si hi ha un topic concret.

Al vostre terminal, escriviu **rostopic list** i comproveu si hi ha un tema anomenat **/counter**.

T3: `$ rostopic list | grep /counter`

Amb aquesta instrucció enumereu tots els topics que s'executen ara mateix i heu filtrat amb l'ordre grep els que contenen la paraula **/counter**. Si apareix, el topic vol dir que s'està executant com hauria de ser.

Podeu sol·licitar informació sobre un topic fent la informació de **rostopic <nom_del_topic>**.

Ara, escriuiu **rostopic info /counter**.

T3: `$ rostopic info /counter`

Haurieu de tenir una sortida com la següent:

```
user ~ $ rostopic info /counter
Type: std_msgs/Int32

Publishers:
* /topic_publisher (http://ip-172-31-16-133:47971/)

Subscribers: None
```

La sortida indica el tipus d'informació publicada (**std_msgs/Int32**), el node que està publicant (**/topic_publisher**) i si hi ha un node que escolta aquesta informació (Cap en aquest cas).

Ara, escriuiu **rostopic echo /counter** i comproveu la sortida del tema en temps real:

Hauríeu de veure una successió de números consecutius, semblant al següent:

```
rostopic echo /counter
data:
985
---
data:
986
---
data:
987
---
data:
988
---
```

Expliquem amb més detall el què ha passat. Primer, esmicolem el codi que hem executat. Podeu consultar els comentaris al codi següent per explicar què fa cada línia del codi:

```
#include <ros/ros.h>
#include <std_msgs/Int32.h>
// Import all the necessary ROS libraries and import the Int32 message from
```

```

the std_msgs package

int main(int argc, char** argv) {

    ros::init(argc, argv, "topic_publisher"); // Initiate a Node named
'topic_publisher'
    ros::NodeHandle nh;

    ros::Publisher pub = nh.advertise<std_msgs::Int32>("counter", 1000);
//      Create      a      Publisher      object,      that      will
// publish on the /counter topic messages
// of type Int32
    ros::Rate loop_rate(2); // Set a publish rate of 2 Hz

    std_msgs::Int32 count; // Create a variable of type Int32
    count.data = 0; // Initialize 'count' variable

    while (ros::ok()) // Create a loop that will go until someone stops the
program execution
    {
        pub.publish(count); // Publish the message within the 'count'
variable
        ros::spinOnce();
        loop_rate.sleep(); // Make sure the publish rate maintains at 2 Hz
        ++count.data; // Increment 'count' variable
    }

    return 0;
}

```

Bàsicament, el que fa aquest codi és **iniciar un node i crear un publisher que segueixi publicant al tema "/counter" una seqüència d'enters consecutius**. Resumint:

Un publisher és un node que continua publicant un missatge en un topic. Així que ara... què és un topic?

Un topic és un canal que conté informació, on altres nodes de ROS poden publicar o llegir informació. Vegem ara algunes ordres relacionades amb topics (algunes d'elles ja les heu utilitzat).

Per obtenir una llista de topics disponibles en un sistema ROS, heu d'utilitzar la següent comanda:

```
rostopic list
```

Per **llegir la informació que es publica en un topic**, utilitzeu l'ordre comanda:

```
rostopic echo <topic_name>
```

Aquesta ordre començarà a imprimir tota la informació que s'està publicant al topic, que de vegades pot resultar molest (és a dir: quan hi ha una quantitat massiva d'informació o quan els missatges tenen una estructura molt gran). En aquest cas, podeu llegir només l'últim missatge publicat en un topic amb la següent comanda:

```
rostopic echo <topic_name> -n1
```

Per **obtenir informació sobre un topic determinat**, utilitzeu la següent comanda:

```
rostopic info <topic_name>
```

Finalment, podeu comprovar les diferents opcions que té l'ordre rostopic mitjançant l'ordre següent:

```
rostopic -h
```

Nota: Quan acabeu amb aquesta secció, assegureu-vos **d'ATURAR** el codi executat anteriorment.

3. Messages

Com haureu notat, els topics gestionen la informació mitjançant missatges. Hi ha molts tipus diferents de missatges.

En el cas del codi que heu executat abans, el tipus de missatge era **std_msgs/Int32**, però ROS proporciona molts missatges diferents. Fins i tot podeu crear els vostres propis missatges, però es recomana utilitzar els missatges predeterminats de ROS quan sigui possible.

Els missatges es defineixen en fitxers **.msg**, que es troben dins d'un directori **msg** d'un paquet. Per **obtenir informació sobre un missatge**, feu servir la següent comanda:

```
rosmmsg show <message>
```

Per exemple, intentem obtenir informació sobre el missatge **std_msgs/Int32**. Escriviu l'ordre següent i comproveu la sortida.

T1:

```
$ rosmmsg show std_msgs/Int32
```

Obtindreu la següent sortida

```
[std_msgs/Int32]:  
int32 data
```

En aquest cas, el missatge **Int32** només té una variable anomenada **data** de tipus **int32**. Aquest missatge **Int32** prové del paquet **std_msgs**, i el podeu trobar al seu directori **msg**. Si voleu, podeu fer una ullada al fitxer **Int32.msg** executant l'ordre següent:

T1: `$ roscd std_msgs/Int32`

Exemple (aquest exercici el va fer a IR, podeu recuperar el codi i comprovar com funciona):

- Modifiqueu el codi que heu utilitzat anteriorment perquè ara publiqui dades al topic **/cmd_vel**.
- Torneu a compilar el teu paquet.
- Inicieu el programa i comproveu que el robot es mou, per a veure què el robot es mou heu de llançar els següents paquets:
 - Al robot:
 - `$ ssh ubuntu@{IP_ADDRESS_OF_RASPBERRY_PI}`
 - `$ roslaunch turtlebot3_bringup turtlebot3_robot.launch`
 - Al pc remot:
 - El vostre node

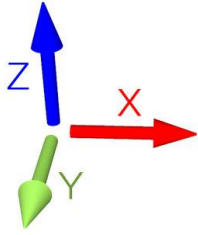
Pistes:

1.- El topic **/cmd_vel** és el topic utilitzat per moure el robot. Feu un **rostopic info /cmd_vel** per obtenir informació sobre aquest topic i identificar el missatge que utilitza. Heu de modificar el codi per utilitzar aquest missatge.

2.- Per omplir el missatge **Twist**, cal crear una instància del missatge. En C++, això es fa així: **geometry_msgs::Twist var;**

3.- Per conèixer l'estructura dels missatges **Twist**, cal utilitzar l'ordre **rosmmsg show**, amb el tipus de missatge que utilitza el topic **/cmd_vel**.

4.- En aquest cas, el robot utilitza un connector d'accionament diferencial per moure's. És a dir, el robot només es pot moure linealment a l'eix x, o rotacionalment a l'eix angular z. Això vol dir que els únics valors que necessiteu per omplir el missatge **Twist** són la x lineal i la z angular.



5.- Les magnituds del missatge Twist estan en m/s, per això es recomana utilitzar valors entre 0 i 1. Per exemple, 0'5 m/s.

4. Topic Subscriber

Heu après que un topic és un canal on els nodes poden escriure o llegir informació. També heu vist que podeu escriure sobre un topic mitjançant un editor, de manera que potser penseu que també hi hauria d'haver algun tipus d'eina similar per llegir informació d'un topic. D'això se'n diu **subscriber**. Un subscriber és un node que llegeix informació d'un topic. Executeu el següent codi:

Exemple 1:

- Creeu un paquet nou anomenat **topic_subscriber_vostrenom_pkg**. Quan creeu el paquet, afegiu com a dependències **roscpp** i **std_msgs**.
- Dins de la carpeta **src** del paquet, creeu un fitxer nou anomenat **simple_topic_subscriber_vostrenom.cpp**. Dins d'aquest fitxer, copieu el contingut de `simple_topic_subscriber.cpp`
- Creeu un fitxer de launch per llançar aquest codi.
- Feu les modificacions necessàries al fitxer `CMakeLists.txt` i compileu el paquet.

```
#include <ros/ros.h>
#include <std_msgs/Int32.h>

void counterCallback(const std_msgs::Int32::ConstPtr& msg)
{
    ROS_INFO("%d", msg->data);
}

int main(int argc, char** argv) {

    ros::init(argc, argv, "topic_subscriber");
    ros::NodeHandle nh;

    ros::Subscriber sub = nh.subscribe("counter", 1000, counterCallback);
```

```
    ros::spin();  
  
    return 0;  
}
```

Al CMake heu d'afegir:

```
add_executable(simple_topic_subscriber_vostrenom  
src/simple_topic_subscriber_vostrenom.cpp)  
  
add_dependencies(simple_topic_subscriber_vostrenom  
${simple_topic_subscriber_vostrenom_EXPORTED_TARGETS}  
${catkin_EXPORTED_TARGETS})  
target_link_libraries(simple_topic_subscriber_vostrenom  
    ${catkin_LIBRARIES}  
)
```

Aneu al vostre terminal i escriviu el següent:

T3: `$ rostopic echo /counter`

Hauríeu de veure una sortida com aquesta, o simplement no veure res en la pantalla:

```
WARNING: no messages received and simulated time is active.  
Is /clock being published?
```

Aquest warning vol dir que ningú publica al topic **/counter**, de manera que no hi ha informació per llegir. Aleshores, publiquem alguna cosa sobre el topic i veurem què passa. Per això, introduïm una nova comanda:

```
rostopic pub <topic_name> <message_type> <value>
```

Aquesta ordre publicarà el missatge que especifiqueu amb el valor que especifiqueu, al topic que especifiqueu.

Obriu un altre terminal (deixeu el que té l'eco rostopic obert) i escriviu la següent comanda:

T4: `$ rostopic pub /counter std_msgs/Int32 15`

Ara comproveu la sortida de la consola on heu tornat a fer l'eco rostopic. Hauríeu de veure alguna cosa com això: .

```
data:
15
---
```

El vostre programa de subscriptors ha rebut el valor que heu publicat (que imprimeix el valor a la pantalla).

Ara comproveu la sortida del shell on heu executat el vostre codi de subscriptor:

```
NODES
/
  topic_subscriber (topic_subscriber_pkg/simple_topic_subscriber)
ROS_MASTER_URI=http://localhost:11311

core service [/rosout] found
process[topic_subscriber-1]: started with pid [28907]
[ INFO] [1515034819.345108919, 12460.180000000]: 5
```

Abans d'explicar-ho tot amb més detall, expliquem el codi que heu executat.

```
#include <ros/ros.h>
#include <std_msgs/Int32.h>

void counterCallback(const std_msgs::Int32::ConstPtr& msg) // Define a
function called 'callback' that receives a
// parameter named 'msg'
{
    ROS_INFO("%d", msg->data); // Print the value 'data' inside the 'msg'
parameter
}

int main(int argc, char** argv) {

    ros::init(argc, argv, "topic_subscriber"); // Initiate a Node called
'topic_subscriber'
    ros::NodeHandle nh;

    ros::Subscriber sub = nh.subscribe("counter", 1000, counterCallback); //
Create a Subscriber object that will
// listen to the /counter topic and will
//
// call the 'callback' function each time
// it reads something from the topic

    ros::spin(); // Create a loop that will keep the program in execution

    return 0;
}
```

Bàsicament s'ha creat un node subscriber que escolta un topic amb nom **/counter**, i cada vegada que llegeix alguna cosa, crida a una funció que fa una impressió del missatge. Al principi, no passa res perquè ningú publicava al topic **/counter**, però quan heu executat l'ordre `pub rostopic`, heu publicat un missatge al topic **/counter**, de manera que el vostre subscriptor ha imprès aquest número i també podríeu veure aquest missatge al *rostopic echo output*.

Ara fem uns exercicis per posar en pràctica el que has après

exemple: Modifiqueu el codi anterior per imprimir l'odometria del robot.

- El robot publica l'odometria del robot al topic **/odom**.
- Haureu d'esbrinar quin missatge utilitza el topic **/odom** i com és l'estructura d'aquest missatge.
- Recordeu compilar de nou el vostre paquet per actualitzar el vostre executable.
- Si trebal·leu en simulació recordeu llençar el paquet de gazebo (pràctica 1).
- Si trebal·leu amb el robot real:
 - Al robot:
 - `$ ssh ubuntu@{IP_ADDRESS_OF_RASPBERRY_PI}`
 - `$ roslaunch turtlebot3_bringup turtlebot3_robot.launch`
 - Al pc remot:
 - El vostre node

Exercici 1 (Entregable)

Creeu un codi perquè un robot sigui capaç d'evitar obstacles mentre navega.

Per ajudar-vos a aconseguir-ho, dividim el projecte en unitats més petites:

- Creeu un Publisher que escrigui al topic **/cmd_vel** per tal de moure el robot.
- Creeu un Subscriber que llegeixi el topic **/scan**. Aquest és el topic on el làser publica les seves dades.
- En funció de les lectures que rebeu del topic del làser, haureu de canviar les dades que esteu enviant al topic **/cmd_vel**, per tal d'evitar el mur. Això vol dir, utilitzar els valors del làser per decidir.

Pistes:

PISTA 1: Les dades que es publiquen al topic **/scan** tenen una gran estructura. Per a aquest projecte, només heu de parar atenció a la matriu "rangs".

Per saber el tipus de missatge que és **/scan** podeu fer:

```
$ rostopic info /scan
Type: sensor_msgs/LaserScan
Publishers:
* /gazebo (http://172.16.0.123:34233/)
```

Subscribers:

```
* /turtlebot3_slam_gmapping (http://172.16.0.123:38381/)  
* /rviz (http://172.16.0.123:37259/)
```

Podeu veure que és del tipus `sensor_msgs/LaserScan`. Per comprovar la informació que conté heu de fer el següent.

```
$ rosmmsg show sensor_msgs/LaserScan  
std_msgs/Header header  
  uint32 seq  
  time stamp  
  string frame_id  
float32 angle_min  
float32 angle_max  
float32 angle_increment  
float32 time_increment  
float32 scan_time  
float32 range_min  
float32 range_max  
float32[] ranges <-- Use only this one  
float32[] intensities
```

PISTA 2: La matriu "ranges" té molts valors. Els que estan al principi de la matriu representen les distàncies que el làser està detectant just davant seu.

PISTA 3: L'abast del làser és d'uns 360 graus començant pel davant.

Forward: `ranges[0]` or `ranges[360]`

Backward: `ranges [180]`

PISTA 4: El làser té un abast de 3 m. Quan obteniu lectures de valors al voltant de 3, vol dir que el làser no detecta res. Si obteniu un valor inferior a 3, això significarà que el làser està detectant algun tipus d'obstacle en aquesta direcció. Quan el làser no detecta cap obstacle en el seu camí, retorna un valor `inf` (infinit).